HAW HAMBURG

Faculty EMI
Center for Microelectronics and Embedded Software
Microcontroller Lab

Lab Session 1

Introduction to Code Composer Studio und GPIO

Table of Contents

1. Objectives of the Laboratory Experiment

This experiment is intended to give you an initial insight into the relationship between hardware and software in embedded systems. In the first part of the experiment, you will familiarize yourself with the embedded programming IDE, and in the second part, you will use the general input and output (GPIO) ports of the TM4C1294 microcontroller. To operate these ports, you will use the hexadecimal keypad, as well as the BCD display described in the appendix.

2. Preparation

2.1 Installing Code Composer Studio (CCS) 12.08.xx

Lab computers already have the IDE installed. For installation on your own PC (not required, but recommended), please follow the instructions from the Moodle course.

2.2 Functionality of the Hexadecimal Keypad and BCD Display

Familiarize yourself with the functionality of the hexadecimal keypad and the BCD display, which you will use later in the experiment (see appendix).

2.3. Preparation Checklist

- ☐ I have read the experiment instructions thoroughly and have an idea of how to implement them.
- ☐ (I have installed CCS.)
- ☐ I understand how the hexadecimal keypad and BCD display work and the necessity of the delay function.
- ☐ I know where to find the necessary registers for configuring the GPIO ports and how to access them (see lecture!).

3. Lab Session Part 1: Code Composer Studio Tutorial

Prerequisites:

- You have installed Code Composer Studio version 12.8.x (see instructions in Moodle) or are working on a lab computer.
- You have downloaded and installed the additional material (Tivaware) from Moodle (also included in the instructions).
- You have downloaded the default project and placed it in the folder C:\ti\ (or /home/user/ti/) (but not unpacked).

3.1 Creating a Project

To create a new project, follow these steps:

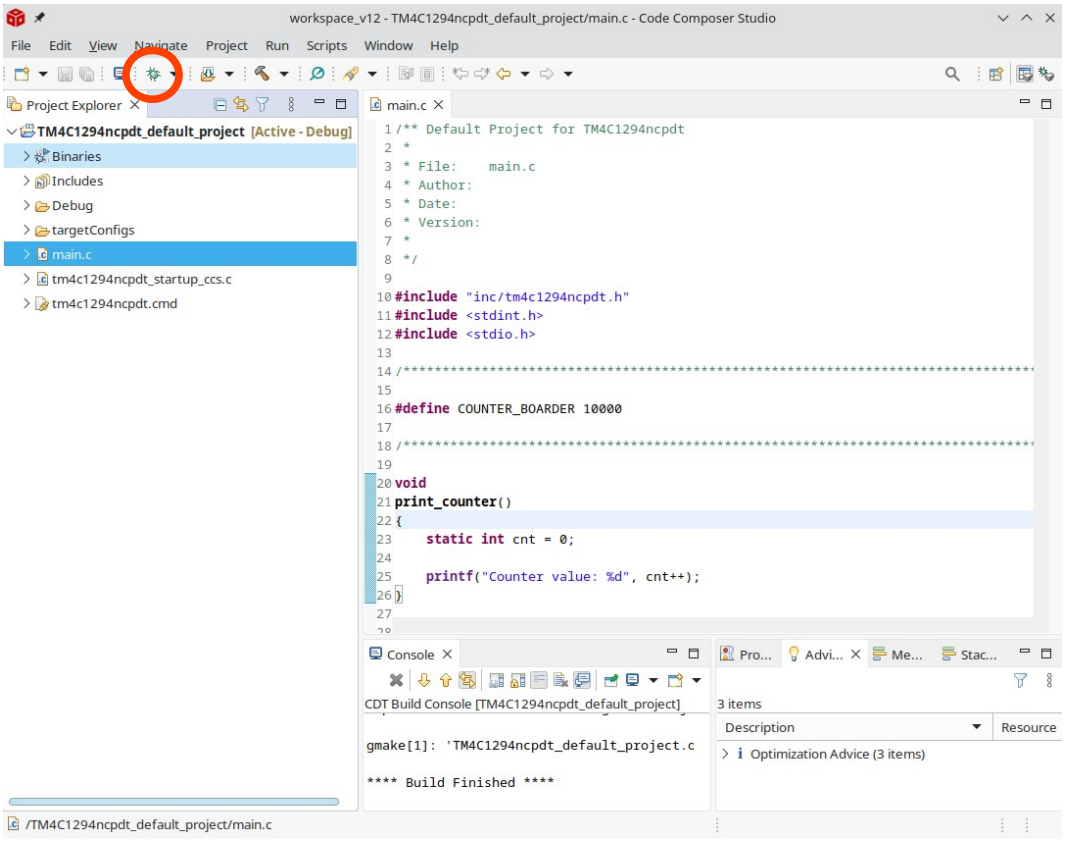
1. File → Import → C/C++ → Select CCS Projects and confirm with Next
2. Select Import from Archive File and click Browse
3. Navigate to the downloaded default project template (e.g., C:\ti\ TM4C1294ncpdt_default_project.zip), select it and click Open
4. Click Finish to create the project
5. Right-click the project in the Project Explorer to rename it
6. Verify the project was created successfully by compiling it with Ctrl + B or the hammer icon. There should be no errors or warnings
7. The new project should now be saved in the default workspace (C:\Users\...\ workspace_v12\)

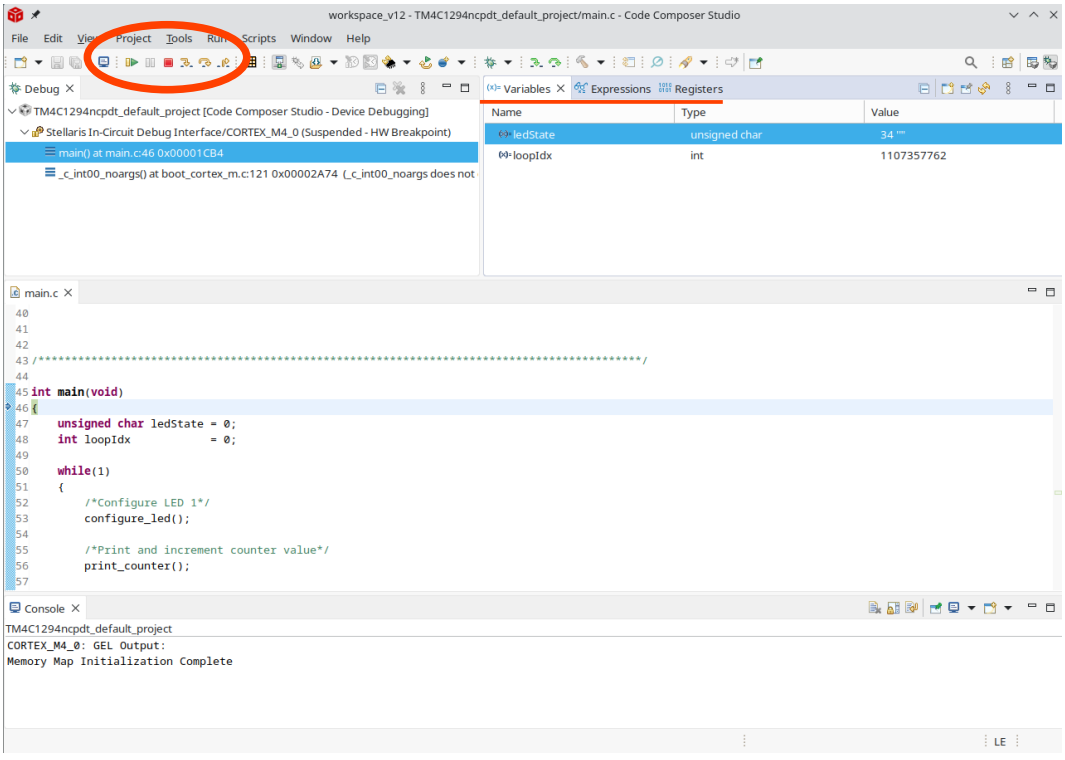
3.2 Debugging the Project

Please connect the microcontroller board (debug port) via USB to the PC.

3.2.1 Starting a New Debug Session

1. Start the debug session with F11 or the green bug icon
2. Wait until the project is compiled and the board's program memory is flashed
3. Start program execution with F8 or the green Resume icon
4. If you started the default project, LED D1 should now blink





3.2.2 Pausing Program Execution and Reading Variables and Registers

Use the Suspend icon to pause execution. In the Variables tab, you can read the current values of your variables. This is especially useful if your program behaves unexpectedly.

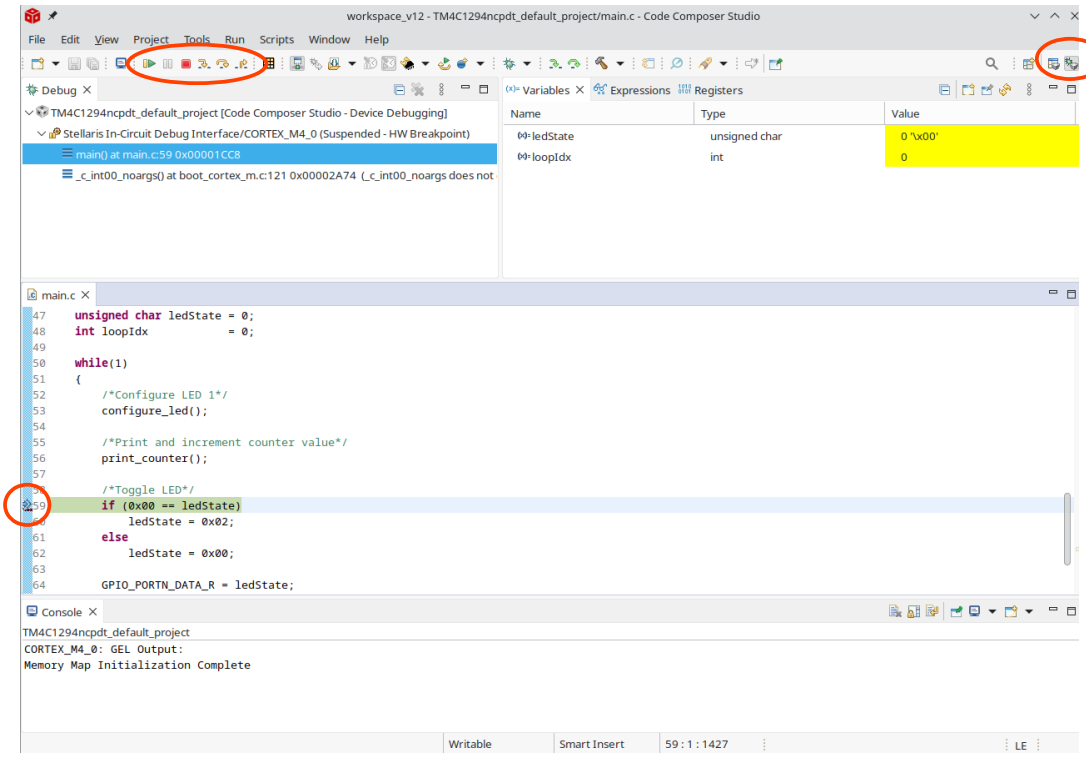
Task: Pause the program and answer the following questions:

1. Which line of code did you stop at?
2. What is the value of the variable *loopIdx*?
3. Navigate to the Register tab. What is the value of the register GPIO_PORTN_DATA_R? Is the LED currently on or off?

3.2.3 Setting Breakpoints and Step-by-Step Execution

You can use breakpoints to pause your program when it reaches a specific instruction / line. It is also possible to execute the program line by line.

- You can set breakpoints by double-clicking the blue bar next to a line
- Use Step Over and Step Into to execute the program line by line



3.2.3 Terminating the Debug Session

Use the red button to terminate the debug session and return to the edit view of CCS. You can also manually switch between views using the top-left menu.

If you want to make changes to your program, you must always end the current debug session and start a new one so that your changes are flashed to the board.

4. Lab Session Part 2: Hexadecimal Keypad and GPIO Introduction

In the following, you will learn how to configure GPIO ports using the keypad.

- Refer to the examples from the lecture for GPIO register setup.
- Don't try to meet all requirements right from the start!
- Instead, think about which (minimal) functionality you want to implement first and expand it gradually. Save your working intermediate results!

4.1 Program for reading a key on the hexadecimal keypad

Write a C program that uses an infinite loop to determine which keys on the hexadecimal keypad are pressed. The result should be output using *printf*(..)!

Requirement 1: Each key should only be displayed once as long as the key is pressed.

Requirement 2: If several keys are pressed at the same time, an error message should be output.

4.2 Displaying the key on a 7-segment (BCD) display

Extend the program from 4.1 so that the currently pressed key is displayed on the 7-segment display. The function of the 7-segment display can be found in the appendix.

Implementation notes

- Take into account the runtime delay caused by the keypad between input and output.
- Implement your program step by step and integrate the requirements one after the other.
- The program should be designed so that the parts can be reused in other experiments.

Appendix 1: How the keypad works

Use Figures 1 and 2 to familiarize yourself with how the MP Lab's hexadecimal keypad works:

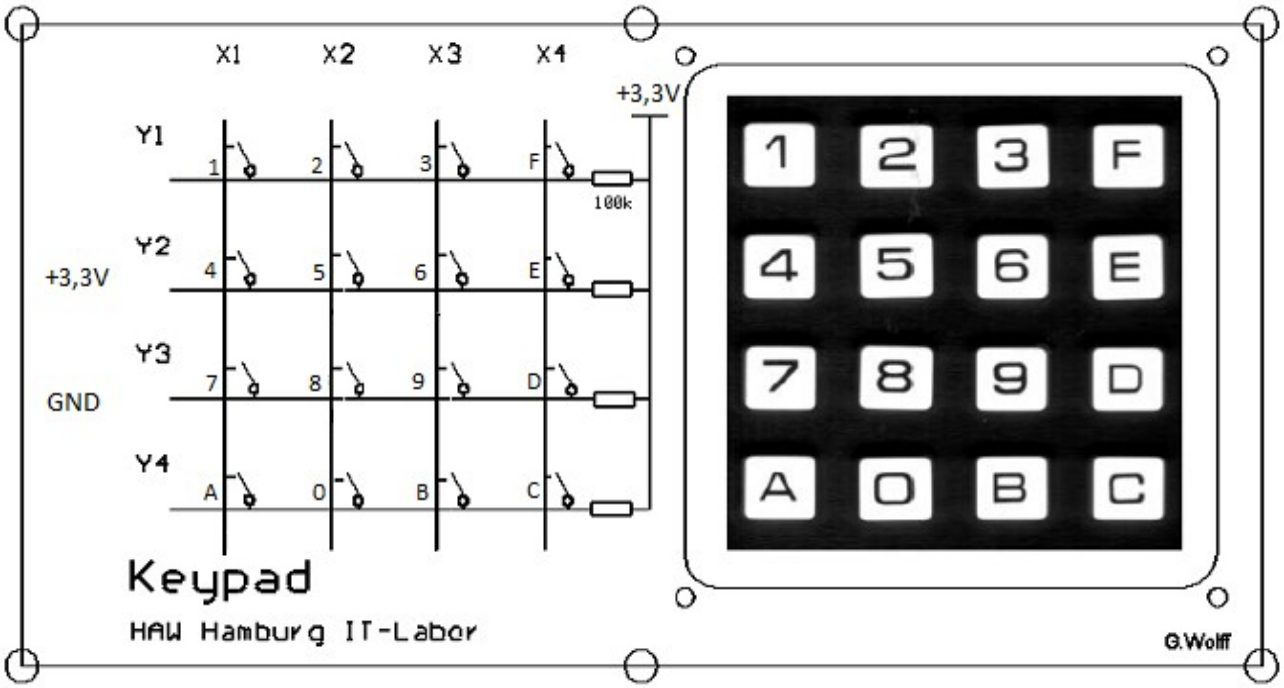


Figure 1: Hexadecimal keypad

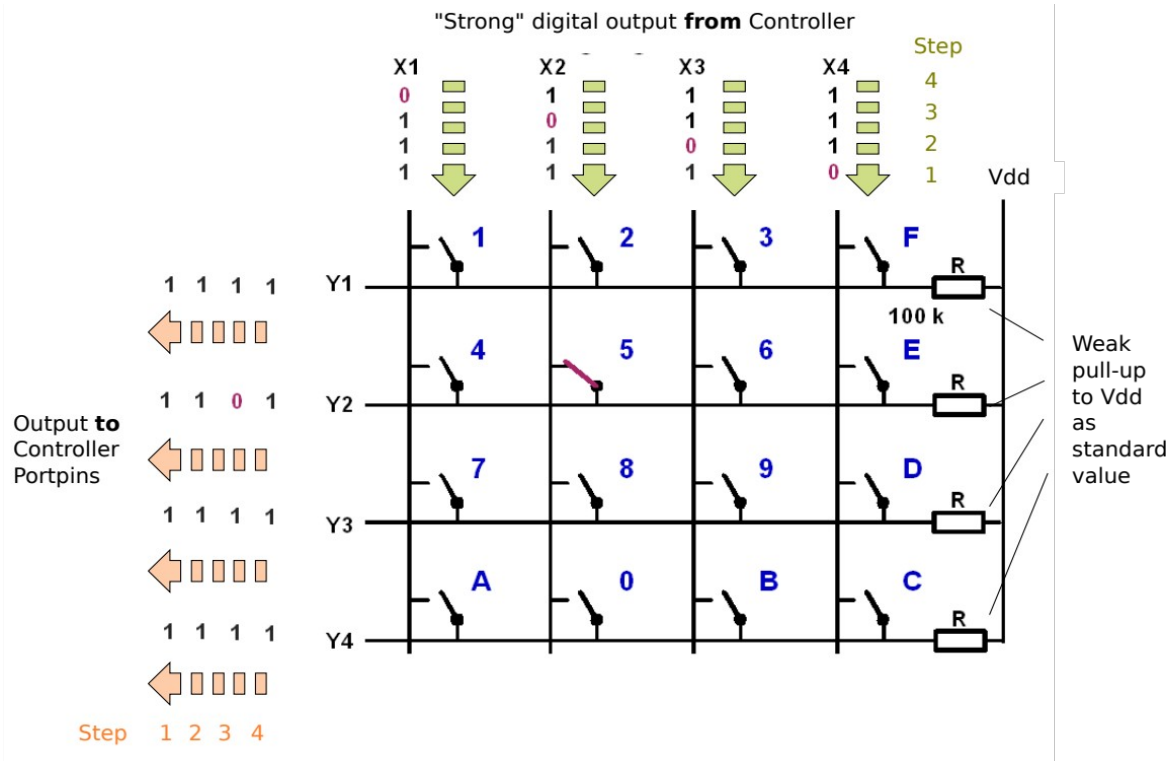


Figure 2: Example of how the hexadecimal keypad works

- The 4 inputs X1 to X4 of the keypad are controlled by the pins of a GPIO port (e.g., port M), which function as outputs.
- The 4 outputs Y1 to Y4 of the keypad are connected to pins (e.g., from port M), which function as inputs.
- In four steps, t1 to t4, each of the keypad inputs is set to ‘0’ in sequence. All other keypad inputs are set to ‘1’.
- The input at which ‘0’ is present and the output at which ‘0’ appears can be used to determine which key was pressed.

In Figure 2, the “5” key was pressed as an example because at time t1, the ‘0’ from input X2 appears at output Y2.

Runtime delays

Due to the high processing speed of microcontrollers, runtime delays between the input and output of external hardware must be taken into account. The example in Figure 3 shows a voltage jump from the high level (logical value 1) to the low level (logical value 0).

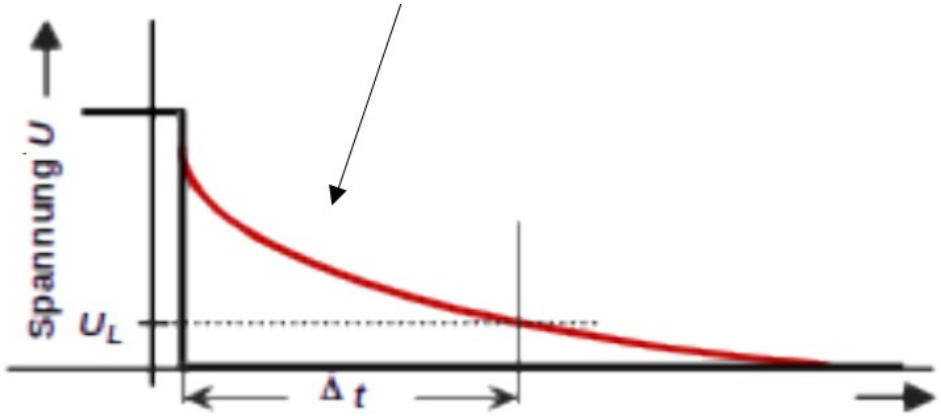


Figure 3: Runtime delay

Due to the capacitive load in the test setup, the voltage at the module follows an exponential function (shown in red), so that the low level UL is only reached after a time delay. Additional signal delays can be caused by active modules such as driver circuits.
To compensate for these external signal propagation times, microcomputer programs often have to delay the evaluation until the levels are sufficiently stable. Wait functions are used for this purpose.

Waiting functions are often implemented with an integrated timer. As long as the use of the timer is not known, a wait loop can be implemented, for example, with the following for loop in the “wait” function:

```
void wait (unsigned int steps)
{
   unsigned int j;
   for (j = 0;  j < steps;  j++ );
}
```

The execution time of a program loop in C depends on the conversion into machine commands and the clock speed of the controller. This time is generally not directly known, causing this method to be unprecise.

Appendix 2: How the 7-segment display (BCD display) works

BCD stands for Binary Code Decimal. It represents a hexadecimal value (0 to 9, A to F) with an input value of 4 bits. Therefore, two digits are represented with 8 data lines.
Data lines D0-D3 represent the digits in the rear position of the display, while lines D4-D7 represent the digits in the front position (you only need the first 4 data lines for the task).

The control of the individual modules using the enable inputs is shown in Figure 3 using the example of 1.29 V. The respective EN line is used to select the desired module for which the data lines supply new values.

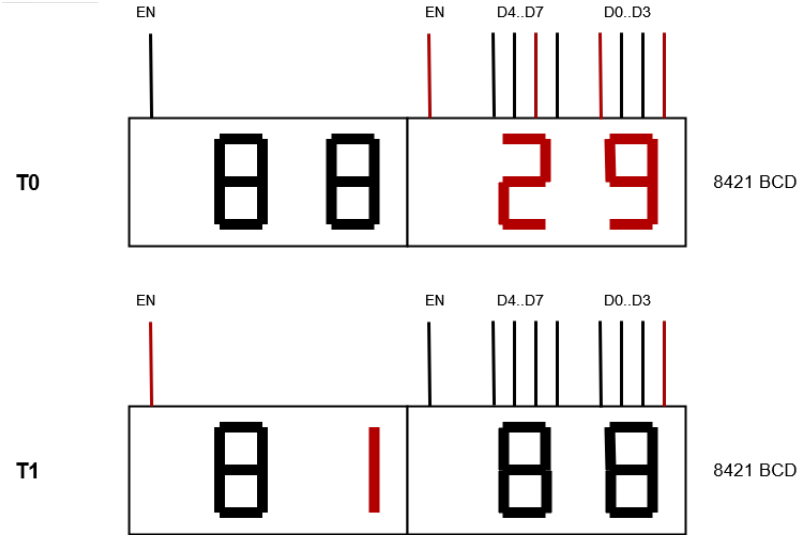


Figure 4: Example of controlling the BCD display